

Prompting Patterns: A Working Taxonomy

Conversation Log | June 27, 2026

USER

create a list of prompting patterns. Examples are planning, brainstorming, spec-driven development, etc.

CLAUDE

Here's a working taxonomy of prompting patterns, grouped by what they're for. A lot of these will already feel familiar from your SDD setup, since spec-driven development itself is really a composite of several of these.

Structuring the work

- **Decomposition** — break a large task into smaller, ordered subtasks before executing any of them
- **Planning** — have the model produce a plan/outline first, get approval, then execute (this is basically what /plan does in your SDD workflow)
- **Spec-driven development** — write the spec (intent, constraints, acceptance criteria) before any implementation; the spec becomes the source of truth the rest of the session is checked against
- **Scaffolding** — give the model a skeleton (file structure, function signatures, headers) and have it fill in the body, rather than generating from a blank page

Generating options

- **Brainstorming** — ask for many candidate ideas/approaches before judging any of them, to avoid anchoring on the first plausible answer
- **Tree of thoughts** — generate multiple reasoning branches, evaluate each, and prune — useful when a single chain-of-thought tends to lock in too early
- **Analogical reasoning** — prompt the model to map the problem onto a structurally similar one it already understands

Reasoning quality

- **Chain-of-thought** — ask the model to reason step-by-step before answering
- **Self-critique / critic pattern** — have the model generate an answer, then separately critique its own output against criteria, then revise
- **Chain-of-verification** — after answering, have the model generate verification questions about its own claims and check them
- **Devil's advocate / red-teaming** — explicitly prompt for the strongest counterargument or failure mode before finalizing

Iterative loops

- **Reflexion** — generate, evaluate against a goal or test result, feed the failure back in, retry
- **ReAct (Reason + Act)** — interleave reasoning steps with tool calls/actions, observing results between steps (this is close to how agentic coding sessions actually run)
- **Test-driven prompting** — write the tests/acceptance criteria first, then prompt for an implementation that satisfies them

Shaping the output

- **Few-shot / worked examples** — show 1-3 examples of the input→output mapping you want before the real request
- **Role/persona framing** — assign an expert persona to bias tone, vocabulary, or rigor (“review this as a security auditor”)
- **Template filling** — give a fixed output structure (like your constitution/spec/plan/tasks templates) and have the model populate it rather than free-form generate
- **Constraint-based prompting** — front-load hard constraints (language version, style guide, forbidden APIs) so the model self-filters rather than getting corrected after

Review/retrospective patterns

- **Pre-mortem** — before starting, ask “how could this plan fail?” to surface risks early
- **Retrospective / postmortem** — after completion, review what worked, what didn't, and what to change next time (useful for refining something like your `sdd-init.ps1` iteratively)

If you want, I could map which of these your current SDD slash-command setup already encodes implicitly and which ones might be worth adding as explicit commands (e.g., a `/critique` or `/premortem` step).

USER

yes, do that

CLAUDE

Good, that confirms the structure: `sdd:init`, `sdd:feature`, `sdd:plan`, `sdd:implement` — four commands, three files per feature (spec/plan/tasks), one constitution per repo. Here's the mapping.

Already encoded, just under different names

Pattern	Where it lives in your SDD setup
Spec-driven development	The whole system — <code>spec.md</code> is the named pattern
Template filling	<code>constitution/spec/plan/tasks</code> templates, populated with placeholders
Decomposition	<code>sdd:plan</code> breaking the spec into phased, ordered tasks

Constraint-based prompting	constitution.md — naming conventions, folder layout, error handling rules every later step must obey
Test-driven prompting	“Checkpoints” in tasks.md that verify a clean build after each phase
Scaffolding	sdd:plan naming concrete files/classes/functions before sdd:implement writes a line of code
ReAct-style loop	sdd:implement walking the checklist top to bottom, marking [x], build-checking at each checkpoint

Implicit but not yet a named step worth calling out

- **Pre-mortem** — sdd:plan step 4 says “flag any decisions that deviate from the constitution or any risks noted in the plan,” but it’s a side-effect of planning rather than its own pass. Worth promoting to an explicit step that asks “how could this plan fail, and what in the spec might be ambiguous enough to cause rework?” before the plan is marked Approved.
- **Self-critique / critic pattern** — there’s no step where the model reviews its own spec.md or plan.md against the constitution before handing it to you for approval. Right now you’re the only critic.
- **Chain-of-verification** — sdd:implement checks the build compiles at each checkpoint, but doesn’t verify the implementation actually satisfies each Goal/Edge Case listed in the spec, just that it doesn’t break.

Missing entirely — worth adding as commands

1. **/sdd:critique [feature-name]** — run after sdd:feature drafts a spec or after sdd:plan drafts a plan, before you approve it. Has the model argue against its own draft: ambiguous requirements, missed edge cases, plan steps that don’t actually satisfy a stated Goal, naming inconsistent with the constitution. Cheap insurance before you spend approval-cycles catching it yourself.
2. **/sdd:premortem [feature-name]** — run once a plan is drafted but before approval: “assume this plan was implemented and it failed in production/use — what’s the most likely reason?” Different from critique in that critique checks consistency, premortem checks failure modes.
3. **/sdd:retro [feature-name]** — run after sdd:implement marks tasks Complete: what deviated from the plan, what constitution rule was awkward to follow, what should change in the constitution or templates for next time. This is the step that would let your SDD infrastructure itself improve across projects rather than staying static.

The natural order would be: init → feature → critique → plan → premortem → implement → retro. Two new commands (critique, premortem) slot between existing steps; retro closes the loop at the end. If you want, I can draft the actual .md command files for these three, matching the style of your existing four.